

CENTRO ESTADUAL DE EDUCAÇÃO PAULA SOUZA
ESCOLA TÉCNICA ESTADUAL ZONA LESTE
Mtec Desenvolvimento De Sistemas AMS

João Pedro Bispo de Biasi

Lucas Bonfim Vilela

Neluma Siqueira Lopes

BLOOMY: Sistema Interativo Para Amparo Educacional.

São Paulo

2025

João Pedro Bispo de Biasi

Lucas Bonfim Vilela

Neluma Siqueira Lopes

**BLOOMY: Sistema Interativo No Amparo Educacional Do Ensino
Básico.**

Trabalho de Conclusão de Curso apresentado ao
Mtec Desenvolvimento de Sistemas AMS da Etec
Zona Leste, orientado pelo Prof. Esp. Jeferson
Roberto de Lima, como requisito parcial para a
obtenção do título de técnico em Desenvolvimento de
Sistemas.

São Paulo

2025

BLOOMY

Sistema Interativo No Amparo Educacional Do Ensino Básico.

João Pedro Bispo de Biasi

Lucas Bonfim Vilela

Neluma Siqueira Lopes

Aprovada em __/__/____.

BANCA EXAMINADORA

Prof. Esp. Jeferson Roberto de Lima

Universidade do Jeferson

Prof. (Professor avaliador)

Universidade do Avaliador

Prof. (Professor avaliador)

Universidade do Avaliador

RESUMO

Contrary to popular belief, Lorem Ipsum is not simply random text. It has roots in a piece of classical Latin literature from 45 BC, making it over 2000 years old. Richard McClintock, a Latin professor at Hampden-Sydney College in Virginia, looked up one of the more obscure Latin words, consectetur, from a Lorem Ipsum passage, and going through the cites of the word in classical literature, discovered the undoubtable source. Lorem Ipsum comes from sections 1.10.32 and 1.10.33 of "de Finibus Bonorum et Malorum" (The Extremes of Good and Evil) by Cicero, written in 45 BC. This book is a treatise on the theory of ethics, very popular during the Renaissance. The first line of Lorem Ipsum, "Lorem ipsum dolor sit amet.", comes from a line in section 1.10.32.

The standard chunk of Lorem Ipsum used since the 1500s is Y below for those interested. Sections 1.10.32 and 1.10.33 from "de Finibus Bonorum et Malorum" by Cicero are also reproduced in their exact original form, accompanied by English versions from the 1914 translation by H. Rackham

Palavras-Chave: lore; ipsum; lore lore ipsum; ipsum ipsum; ipsum lore.

ABSTRACT

Contrary to popular belief, Lorem Ipsum is not simply random text. It has roots in a piece of classical Latin literature from 45 BC, making it over 2000 years old. Richard McClintock, a Latin professor at Hampden-Sydney College in Virginia, looked up one of the more obscure Latin words, consectetur, from a Lorem Ipsum passage, and going through the cites of the word in classical literature, discovered the undoubtable source. Lorem Ipsum comes from sections 1.10.32 and 1.10.33 of "de Finibus Bonorum et Malorum" (The Extremes of Good and Evil) by Cicero, written in 45 BC. This book is a treatise on the theory of ethics, very popular during the Renaissance. The first line of Lorem Ipsum, "Lorem ipsum dolor sit amet.", comes from a line in section 1.10.32.

Keywords: lore; ipsum; lore lore ipsum; ipsum ipsum; ipsum lore.

LISTA DE ILUSTRAÇÕES

Figura 01 — Exemplo Diagrama de Caso de Uso	14
Figura 02 — Exemplo Diagrama de Sequência	18
Figura 03 — Exemplo Diagrama de Máquina de Estados	19
Figura 04 — Exemplo Diagrama de Atividade	20
Figura 05 — Exemplo Wireframe de Baixa Fidelidade	21
Figura 06 — Exemplo Wireframe de Alta Fidelidade	22
Figura 07 — Exemplo Código em HTML.....	23
Figura 08 — Exemplo Formulário em HTML.....	24
Figura 9 — Exemplo Código em CSS	25
Figura 10 — Exemplo Código em HTML e CSS.....	27
Figura 11 — Exemplo Formulário em CSS	28
Figura 12 — Exemplo Código em JS	29
Figura 13 — Exemplo Código em JS e HTML	29
Figura 14 — Exemplo Formulário em JS	30
Figura 15 — Exemplo código em React	31
Figura 16 — Exemplo Formulário em React.....	32
Figura 17 — Exemplo de Código em React Native	33
Figura 18 — Exemplo de Código em React Native	33
Figura 19 — Exemplo de Código em React Native	34
Figura 20 — Exemplo de Código em React Native	34
Figura 21 — exemplo de Código em React Native	35
Figura 22 — Exemplo de Código em React Native	36
Figura 23 — Exemplo Formulário em React Native.....	37
Figura 24 — Exemplo Código em TS	38
Figura 25 — Exemplo de código em C++	41
Figura 26 — Microcontrolador ESP8266.....	43

LISTA DE ABREVIATURAS E SIGLAS

Application Programming Interface (API)

Unified Modeling Language (UML)

HyperText Markup Language (HTML)

Cascading Style Sheets (CSS)

JavaScript (JS)

TypeScript (TS)

Internet of Things (IoT)

Bluetooth Low Energy (BLE)

Light Emitting Diode (LED)

Central Processing Unit (CPU)

LISTA DE TABELAS

Tabela 1 — Exemplo da Documentação do Caso de Uso	15
---	----

SUMÁRIO

1 INTRODUÇÃO	11
2 REFERENCIAL TEÓRICO	12
2.1 Desamparo Educacional Sentido Nos Alunos do Ensino Básico	12
2.2 Tecnologias utilizadas	13
2.2.1 Design Thinking	13
2.2.2 Unified Modeling Language (UML)	13
2.2.2.1 Diagrama de Casos de Uso	14
2.2.2.2 Documentação dos Casos de Uso	15
2.2.2.3 Diagrama de Sequência	17
2.2.2.4 Diagrama de Máquina de Estados	18
2.2.2.5 Diagrama de Atividade	19
2.2.3 Wireframes	20
2.2.4 HTML	22
2.2.5 CSS	24
2.2.6 JavaScript	28
2.2.7 React	30
2.2.8 React Native	32
2.2.9 TypeScript	37
2.2.10 Banco de dados	38
2.2.11 Firebase	39
2.2.12 Construct	39
2.2.13 C++	40
2.2.14 Internet of Things (IOT)	42
2.2.15 ESP32	42
2.2.16 Modelagem 3D	43
2.2.17 Impressão 3D	43

3 DESENVOLVIMENTO.....	45
4 CONSIDERAÇÕES FINAIS.....	46
REFERÊNCIAS.....	46

1 INTRODUÇÃO

2 REFERENCIAL TEÓRICO

Este capítulo discorre sobre toda a fundamentação teórica pesquisada para basear a construção do projeto.

2.1 Desamparo Educacional Sentido Nos Alunos do Ensino Básico

Um fato sobre o baixo desempenho acadêmico, discute a relação da má vontade de absorver os saberes passados pelo educador, que não ampara as vivências únicas do aluno, dificultando a trilha para uma educação libertadora (FREIRE, 1996). Além disso, sabe-se que a escola é o meio essencial para um futuro de qualidade, mas pouca importância é dada para o aprendizado; muitos estudantes julgam a escola como insatisfatória, sem sentido, cansativa e conflituosa (MARQUES; CASTANHO, 2011).

Segundo Ana Parisotto e Renata Rinaldi (2016), os professores do ensino básico estão com menos respaldo acadêmico, expondo a dificuldade das matérias em serem compreendidas por si só quando há carência de outras formas educacionais. Com isso, de acordo com Samara de Sena *et. al.*, as mudanças do século XXI tornam evidente que a abordagem educacional deve ser revista e jogos epistêmicos é uma aposta promissora (SENA, *et. al.*, 2016).

2.1.1 Jogos sérios

Como dito por Vasconcelos *et. al* (2017), estudiosos buscaram formas de vincular o brincar dos jogos com o aprendizado, chamados "jogos sérios", onde planejam usar de ferramentas lúdicas promovendo habilidades autônomas aos alunos. Segundo Paula (2015), jogos trazem um mundo imersivo — trilha para o subconsciente das pessoas — e qualquer mensagem, para ele é passível de reflexões intimistas pelo jogador, criando e desenvolvendo o pensamento crítico.

Porém, como visto atualmente, a incidência do uso excessivo de aparelhos eletrônicos cresceu, o que leva todo o foco e interesse por tarefas que te desafiam a diminuir em comparação com as atividades contrárias, como os jogos (ESTADO DE MINAS, 2023).

Tal atividade, traz ao cotidiano das crianças uma gama de estímulos, pondo-as num comodismo, devido a recorrentes atividades superficiais e pouco desafiadoras, levando a um baixo rendimento e interesse nas aulas (G1, 2024).

Considerando os fatos, como uma plataforma de jogos-sérios pode amenizar o baixo rendimento escolar?

2.2 Tecnologias utilizadas

Para a confecção do Bloomy, foram usadas tecnologias pensadas para a solução do problema proposto. A seguir, teremos o referencial teórico sobre essas ferramentas que serão utilizadas.

2.2.1 Design Thinking

De acordo com Talita Pagani, o design tem a função de desenhar um caminho para resolução de um problema — de maneira humana e com atenção individual acima das ferramentas de resolução (PAGANI, 2018).

Como visto na Revista eletrônica de biblioteconomia e ciência da informação, Design Thinking é um método de inovação com a função de entender e solucionar problemas de forma criativa em diversas condições. (FEDERAL DE SANTA *et al.*, 2022).

É apontado no livro Design Thinking: Inovação em Negócios:

Ao desafiar os padrões de pensamento, comportamento e de sentimento 'Design Thinkers' produzem soluções que geram novos significados e que estimulam os diversos aspectos (cognitivo, emocional e sensorial) envolvidos na experiência humana. (VIANNA *et al.*, 2012, p. 14).

Em serviços que lidam com pessoas, o modelo tradicional de resolução de problemas, sistemático e objetivo, não é adequado, é necessário se moldar às necessidades, emoções e desejos de cada indivíduo para entrega de requisitos (PAGANI, 2018).

2.2.2 Unified Modeling Language (UML)

É uma linguagem — uma ferramenta de modelagem — que visa analisar e projetar sistemas orientados a objeto, limitando e direcionando os objetivos do projeto, buscando atender o melhor resultado em todas as áreas possíveis (BOOCH; RUMBAUGH; JACOBSON, 2006).

Segundo Guedes (2009), a UML possui o objetivo de auxiliar os engenheiros de software para a construção — caracterização e detalhamento — da aplicação, não sendo exclusiva à apenas eles.

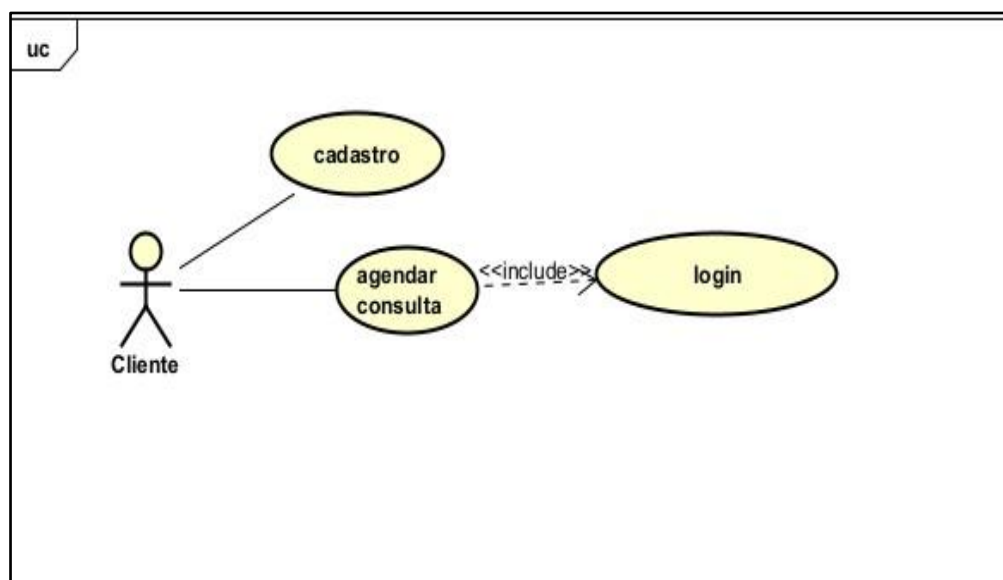
Conforme descrito por Larman (2000), essa ferramenta oferece padrões, que torna possível a visualização e disposição de soluções mais fácil — não prescreve passo a passo, mas oferece estrutura e seriedade ao seu projeto, tornando-o manutenível. Dentro desse modelo, temos padrões normativos para cada especificação do projeto — não necessariamente equivalentes ao código, mas aproximados — não sendo um modelo rígido, apenas um norte do que deve ser feito (FOWLER, 2007).

2.2.2.1 Diagrama de Casos de Uso

Assim como outros diagramas dinâmistas informais, o de Casos de Uso modela a atitude do sistema — como ele deve agir, que comportamento ele precisa ter — envolvendo contexto, e requisitos da aplicação (BOOCH; RUMBAUGH; JACOBSON, 2006).

Como apontado por Guedes (2009), objetiva exemplificar as funcionalidades do sistema, moldando de forma geral, o que ocorrerá — a cadeia de eventos vinculada a cada usuário e função — e baseando outros diagramas. Em outras palavras, converge um ator e os processos que ele desencadeia na aplicação, baseando a construção do software — sendo atores os usuários e os casos de uso às funções — de fácil entendimento prévio da narração dos requisitos (LARMAN, 2000).

Figura 01 — Exemplo Diagrama de Caso de Uso



Fonte: Autoria Própria, 2025

O exemplo acima, demonstra o modelo simples de casos de uso de uma clínica, tendo um ator chamado “Cliente”, e os casos de uso “Cadastro”, “Agendar Consulta” e

“Login”. O Cliente pode interagir com todos os processos, ou seja, ele pode se cadastrar, e em seguida gerenciar seu perfil; é possível que o ator “Cliente” agende uma consulta, mas apenas após efetuar o login, como mostra pela seta tracejada “<<include>>” — elemento condiciona uma ação antes que o caso de uso principal seja realizado.

2.2.2.2 Documentação dos Casos de Uso

Para Booch, Rumbaugh e Jacobson (2005), os comportamentos dos Casos de Uso é o conjunto de ações que são utilizadas para visualizar e documentar o sistema e em função do que foi estimado, convertendo tais requisitos em detalhes fixos.

Contudo, Guedes (200) afirma que a documentação do caso de uso, é a maneira de simplificar as funções gerais expostas no diagrama — tendo o contexto dos processos, os atores que interagem, e as funções que irão ocorrer.

Para Larman (2000), o documento narrativo formal e sucinto — tem as mesmas funções que o diagrama derivado, só que de maneira detalhista, onde aponta as

Tabela 1 — Exemplo da Documentação do Caso de Uso

Nome do caso de uso	Marcar consulta.
Ator principal	Cliente.
Atores secundários	Nenhum.
Resumo	Este caso de uso expressa o fluxo de um cliente marcando uma consulta.
Pré-condições	É preciso efetuar o log-in.
Pós-condições	Uma consulta é cadastrada vinculada ao cliente,
Fluxo principal	

Ações do ator	Ações do Sistema
1. Solicitar log-in.	
	2. Inserir informações do cliente.
Formar a senha da conta para efetuar login.	Consultar cliente com base na senha fornecida e CPF.
página de marcar consulta.	
essa parâmetros para efetuar consulta.	
	7. Inserir consulta.
Restrições/ validações	Para se cadastrar é preciso ser maior de idade. Para marcar uma consulta, é preciso efetuar login.

Fonte: Autoria Própria, 2025

No exemplo, define-se o nome do caso de uso, chamado “Marcar Consulta”.

Então é informado o caso de uso geral, no entanto não há nenhum, então não fora especificado.

Logo, frisado o ator principal e o secundário, respectivamente, é aquele que mais interage com o caso de uso documentado, sendo ele o “Cliente”; não possui, logo não foi especificado. E também um resumo sobre a função desse caso de uso em específico.

Em seguida é definido as pré-condições e pós-condições, as etapas que ocorrem anterior e seguinte ao caso de uso, sendo elas “Efetuar login” e “Cadastrar Consulta” subsequentemente.

Após isso, temos o fluxo principal — onde mostra a cadeia de eventos principais que regem o caso de uso, separado por etapas dos atores e do sistema — como no exemplo, o login é solicitado pelo “Cliente”, que envia suas informações para o sistema inserir e fazer a verificação; com isso é possível fazer o processo de cadastrar consulta, onde o “Cliente” novamente informa seus dados para cadastrar e o sistema além de inserir, retorna à consulta finalizada.

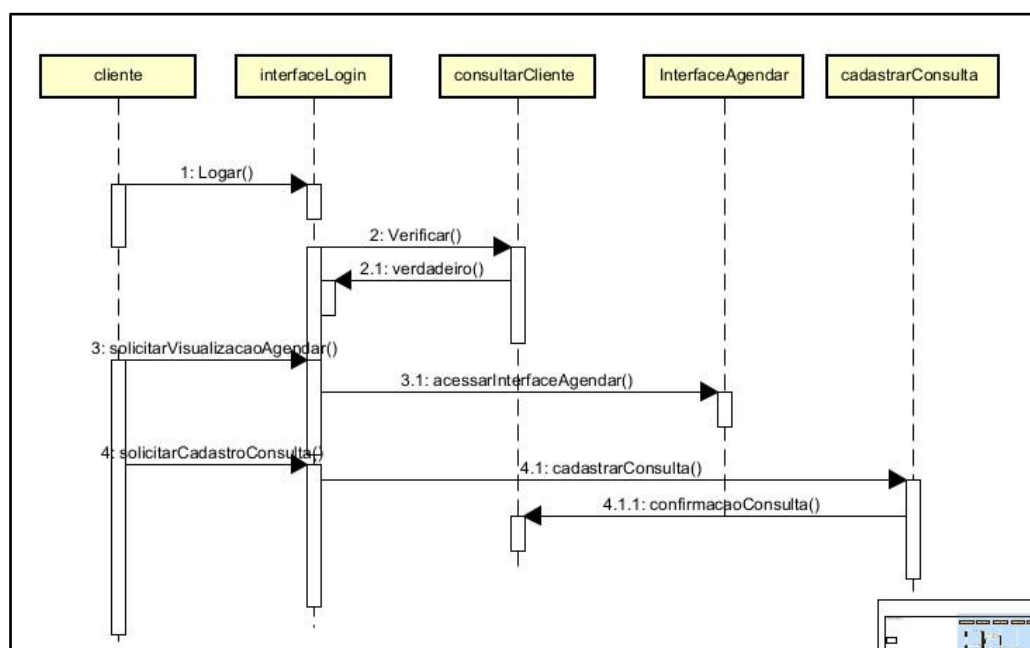
2.2.2.3 Diagrama de Sequência

Segundo Fowler (2007), é a fragmentação de um caso de uso baseada em cenários — o que acontecerá para o usuário em determinada situação — mostrando possíveis resultados para cada situação.

Como explicado por Guedes (2009), é apoiado nos processos desencadeados pelo usuário, determinando o desenrolar do cenário — validando e complementando o diagrama de classes, possibilitando uma aplicação mais sólida e precisa.

Com uma ordenação de mensagens e eventos, o diagrama de sequência dispõe os objetos que reagem e afetam outros, prevendo a duração da interação de usuário com objetos, chamadas de funções e valores de retorno (BOOCH; RUMBAUGH; JACOBSON, 2006).

Figura 02 — Exemplo Diagrama de Sequência



Fonte: Autoria Própria, 2025

Podemos ver neste exemplo, os retângulos de ativação — atores ou funcionalidades ativas no diagrama — sendo eles “Cliente”, “InterfaceLogin”, “consultarCliente”, “interfaceAgendar” e “cadastrarConsulta”. O “Cliente”, na sua linha de vida — linha tracejada abaixo dos retângulos — pode efetuar o login, para ser encaminhado para “interfaceLogin”. Após essa solicitação, os dados fornecidos são levados para “consultarCliente” — onde as informações são validadas — que caso seja verdadeiro, ou seja, esse usuário está cadastrado no sistema, é possível ver outras funcionalidades.

O “Cliente” pode navegar pela “interfaceAgendar”, onde é cabível o cadastro de consulta, que retornará o resultado “confirmacaoConsulta”.

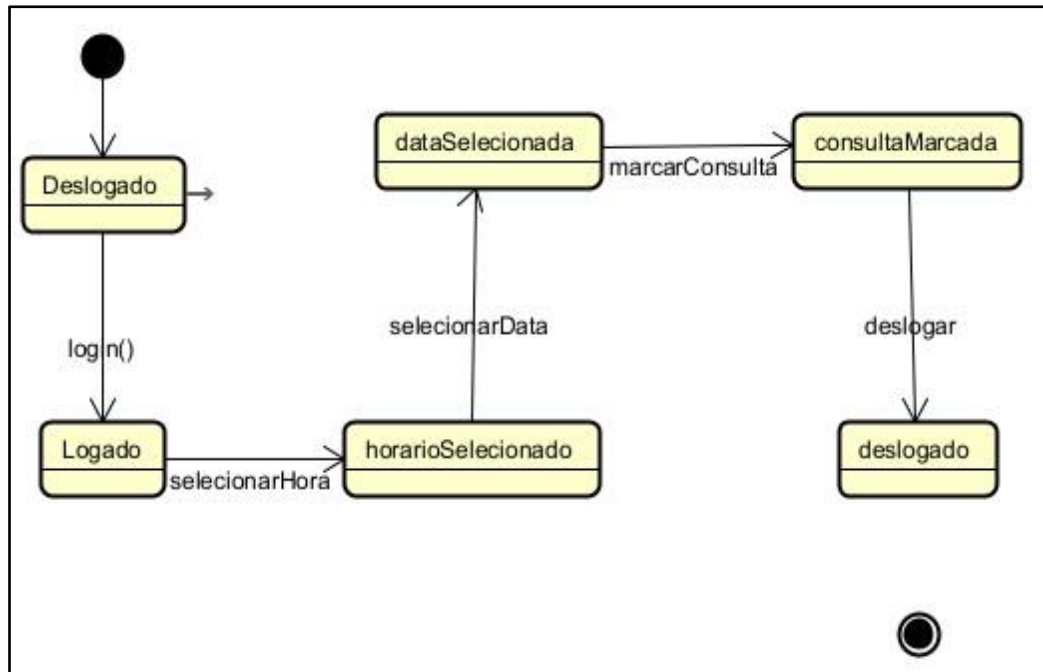
2.2.2.4 Diagrama de Máquina de Estados

De acordo com Fowler (2007), é um diagrama que explana as mudanças de estados em uma máquina — seja devido a imposições ou condições — também definindo uma base de dependência do seu objeto sobre o estado atual.

Usado para modelar as atividades dinâmicas de cada aspecto individual da vida de um sistema, como uma cadeia de eventos determinados pelo estado do objeto — que causará uma ou mais reações (BOOCH; RUMBAUGH; JACOBSON, 2006).

Larman Craig (2000), explica que o diagrama de estados modela apenas o que é necessário, indo de detalhista à simplista de maneira arbitrária — de acordo com os requisitos do sistema.

Figura 03 — Exemplo Diagrama de Máquina de Estados



Fonte: Autoria Própria, 2025

É exemplificado o início do diagrama, marcado pelo círculo preto. Como explicado anteriormente, esse diagrama demonstra estados — retângulos amarelos, que representam a reação após ações das classes e objetos — sendo eles : “Deslogado”, “Logado”, “horarioSelecioneado”, “dataSelecioneada”, “consultaMarcada” e novamente “Deslogado”. As setas indicam eventos, como métodos — ações que afetam os estados — sendo elas “login”, “selecionarHora”, “selecionarData”, “marcarConsulta” e “deslogar”.

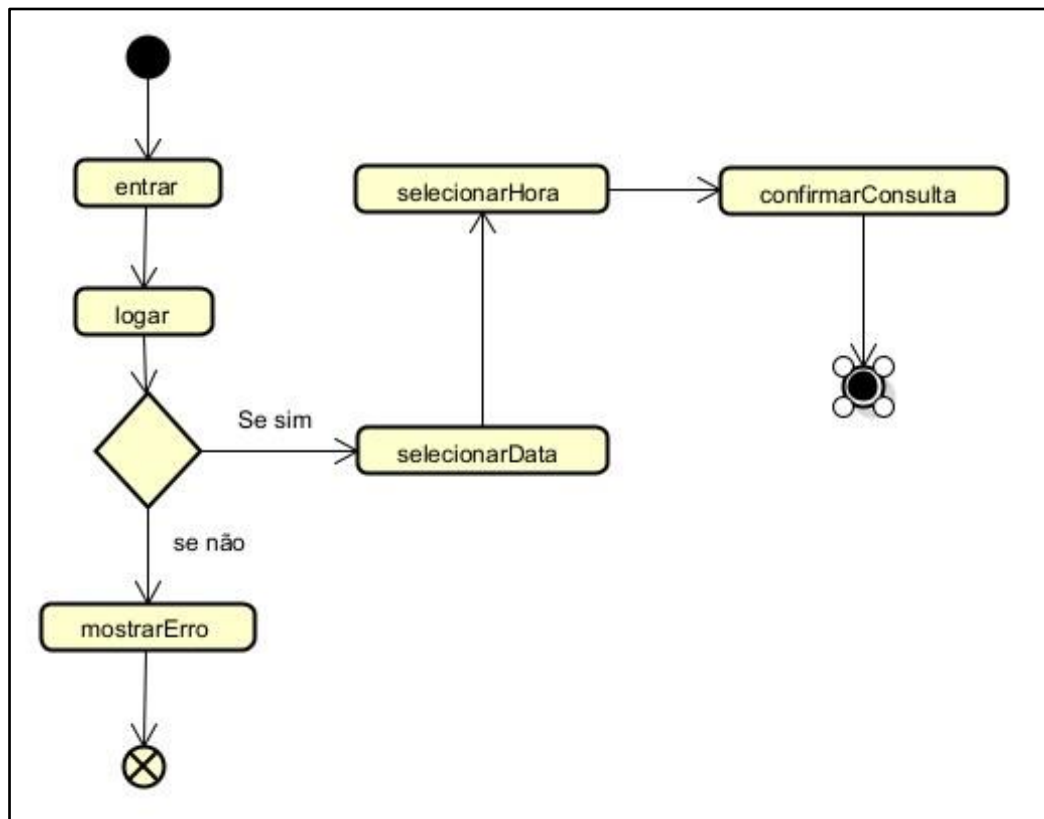
O diagrama é finalizado pelo círculo preto dentro de outro.

2.2.2.5 Diagrama de Atividade

Fowler (2007) expõe, é a ferramenta que indica o fluxo — espécie de fluxograma, onde registra a cadeia de atividades lógicas — simultâneas ou intercaladas, que ocorrerão durante o uso do sistema.

Elucide Guedes (2009), que este modelo enfatiza o curso das operações que serão realizadas pelos métodos — ou seja, um conjunto de ações que expressam as funcionalidades, podendo ou não finalizar a atividade.

Figura 04 — Exemplo Diagrama de Atividade



Fonte: Autoria Própria, 2025

No exemplo acima, o diagrama se inicia com o círculo preto preenchido. segue para as atividades “entrar” e “login”, onde ocorre uma validação, se “login” retorna falso, o sistema mostrar invalidez e fecha atividade; caso retorne verdadeiro, o fluxo continua possibilitando marcar consulta, pelo caminho “selecionarData” e “selecionarHorario”, podendo assim “confirmarConsulta” e encerrar as atividades como um todo.

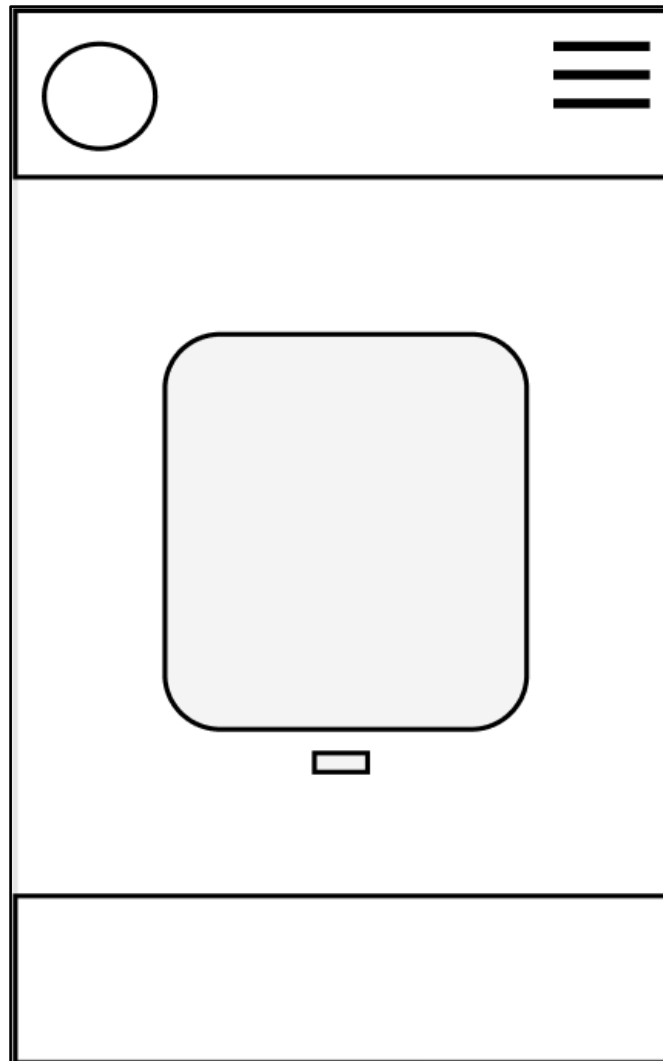
2.2.3 Wireframes

Exemplifica Caio Oliveira, (2021) que wireframes são como as interfaces irão ser montadas — representam a aparência, navegação e funcionalidades básicas do site — tornando mais concretos os eventos feitos em diagramas.

Como desenhos, afirma Fabrício (2014), os wireframes podem ser mais ou menos elaborados, depende de quão perto do design final o produto se encontra, dito isso – wireframes de baixa fidelidade são como rascunhos, passíveis de mudanças.

De acordo com Rogério (2018), os wireframes de alta fidelidade, são os mais próximos de um resultado — quanto mais específico for, menos chances de explorar criativamente se tem, abaixando as possibilidades de uma identidade visual própria.

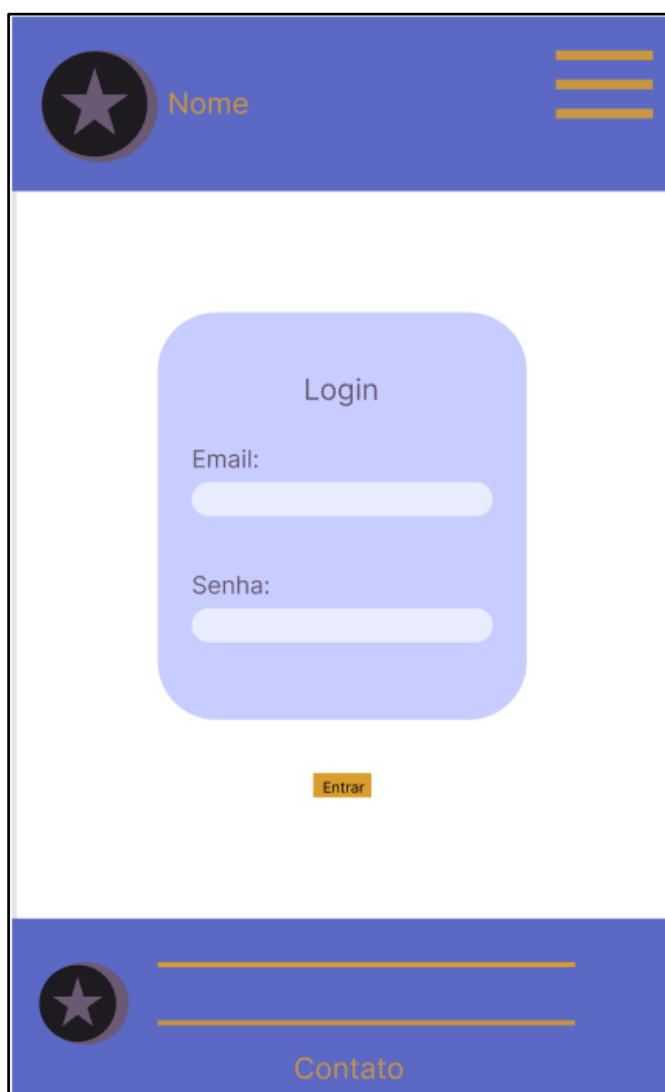
Figura 05 — Exemplo Wireframe de Baixa Fidelidade



Fonte: Autoria Própria, 2025

Para modelar interfaces, pode ser utilizado o Figma, um software que auxilia no processo de desenvolvimento em wireframes, na aparência e na navegabilidade (ACETI *et. al*, 2023).

Figura 06 — Exemplo Wireframe de Alta Fidelidade



Fonte: Autoria Própria, 2025

2.2.4 HTML

De acordo com Fábio Flatschart (2011), o HTML, ou Marcação de Linguagem de Hipertexto (tradução de HyperText Markup Language), é a principal linguagem para construção de páginas na internet.

Segundo Maurício Samy (2008), Hipertexto é todo conteúdo que pode ser inserido em páginas da internet e que possuam a possibilidade de se interligarem com outras aplicações da web, utilizando de links para a sua construção.

Na linguagem HTML, utilizamos de *tags* para a escritura da linguagem, que usam das sintaxes “< >” ao início e “</ >” ao fim de cada linha. Na construção de aplicações web,

as *tags* servem para estruturar como os elementos são inseridos na página (FLATSCHART, 2011).

As principais *tags* utilizadas no HTML para a construção de aplicações são:

- “<!DOCTYPE html>” informa a versão utilizada na marcação ao navegador, permitindo que o mesmo processe a página da melhor forma na internet;
- “<html> & </html>” criado logo após a tag DOCTYPE, esta tag detém todo o código da página, indicando o início e o fim do mesmo;
- “<head> & </head>” está localizado dentro da tag <html>, serve como cabeçalho da aplicação, além de deter as configurações da página;
- “<body> & </body>” está localizado dentro da tag <html>, todo o código dos elementos visíveis na página é escrito dentro desta tag.

A seguir demonstraremos um código em HTML, sendo ele uma interface simples, que exemplifica a construção de um formulário:

Figura 07 — Exemplo Código em HTML

```
1  <!DOCTYPE html>
2  <html>
3
4  <head>
5    <title>Exemplo</title>
6  </head>
7
8  <body>
9    <div>
10      <h1>Formulário</h1>
11      <hr/>
12      <form>
13        <label>Nome:</label>
14        <input/>
15
16        <br/>
17
18        <label>Email:</label>
19        <input/>
20
21        <br/>
22
23        <label>Senha:</label>
24        <input/>
25
26        <br/>
27
28        <button>Enviar</button>
29      </form>
30    </div>
31  </body>
32
33  </html>
```

Fonte: Autoria Própria, 2025

Linha 5: É demonstrado a tag “<title>”, que define o título da página.

Linha 8: Cria uma divisória entre as outras tags por meio da <div>.

Linha 10: Define o tamanho “<h1>” ao texto “Formulário”.

Linha 11: Demarca uma diferença entre a sessão pelo <hr>.

Linha 12: Expressa o início de um formulário.

Linha 13: Define uma legenda para algum campo pela tag “<label>”, como “Nome:”, “Email:” e “Senha:”.

Linha 14: A tag “<input>” expressa um elemento de campo para inserir informações

Linha 16: Dá um espaçamento entre os elementos pela tag “
”.

Linha 28: Inicia um elemento botão chamado “Enviar” pela tag “button”.

A seguir o resultado do exemplo, executado no navegador:

Figura 08 — Exemplo Formulário em HTML

Fonte: Autoria Própria, 2025

2.2.5 CSS

Folhas de Estilo em Cascata, ou apenas CSS, é a linguagem de estilização utilizada para a formatação do layout de páginas. Ele facilita a estilização,

generalizando o código em um arquivo único — economizando tempo de maneira organizada. (JOBSTRAIBIZER, 2009).

Segundo Evandro Miletto e Silvia Bertagnolli (2014), o CSS possibilita a criação de personalizada para títulos, imagens, listas e outros; além da criação de paleta de cores, fontes para texto, alinhamento dos objetos, e outras características relacionada à aparência da aplicação.

Afirma Felipe Scheidt (2015) que o termo “Folha de Estilo” significa um conjunto de regras, que definem a aparência de uma aplicação web — onde as características da página como cores e posições dos objetos são definidas através dessas regras.

A seguir, temos um exemplo de como estilizamos o formulário em HTML que fizemos anteriormente:

Figura 9 — Exemplo Código em CSS

```
1  body {
2      display: flex;
3      justify-content: center;
4      font-family: sans-serif;
5  }
6
7  .caixaFormulario {
8      width: 30%;
9      padding: 10px;
10     border-radius: 10px;
11     background-color: lightblue;
12     text-align: center;
13 }
14
15 .entrada {
16     border: none;
17     margin-bottom: 10px;
18     border-radius: 5px;
19     width: 100%;
20     height: 20px;
21 }
22
23 .botao {
24     border: none;
25     padding: 5px;
26     border-radius: 5px;
27 }
28
```

Fonte: Autoria Própria, 2025

Linha 1: A tag “body” é aberta, onde definimos a estilização geral dos elementos da página.

Linha 2: Definimos o alinhamento dos objetos da página como flexíveis.

Linha 3: Centraliza os elementos dentro do corpo da página.

Linha 4: É definida qual a fonte padrão da página.

Linha 7: Aqui iniciamos a definição de estilização de elemento com a classe “caixaFormulario”. O mesmo acontece na linha 15 e 23, apenas com classes diferentes.

Linha 8: É definida em 30% a largura dos elementos com a classe.

Linha 9: Adicionamos um espaçamento interno de 10 pixels ao redor do conteúdo do elemento.

Linha 10: Arredonda as extremidades do elemento em 10 pixels.

Linha 11: Definimos uma cor de fundo para os elementos da classe, sendo a cor escolhida o azul claro.

Linha 12: Centraliza o texto do elemento.

Linha 16: É removido qualquer borda padrão de elementos da classe “entrada”.

Linha 17: Adicionamos uma margem inferior de 10 pixels no elemento, criando um espaçamento abaixo do mesmo.

Linha 18: Como na linha 10, definimos o arredondamento das bordas do elemento.

Linha 19 e 20: É definida, respectivamente, a largura e altura do elemento.

Figura 10 — Exemplo Código em HTML e CSS

```
1  <!DOCTYPE html>
2  <html>
3
4  <head>
5    <title>Exemplo</title>
6    <link rel="stylesheet" href="exemplo.css">
7  </head>
8
9  <body>
10    <div class="caixaFormulario">
11      <h1>Formulário</h1>
12      <hr/>
13      <form>
14        <label>Nome:</label>
15        <input class="entrada"/>
16
17        <br/>
18
19        <label>Email:</label>
20        <input class="entrada"/>
21
22        <br/>
23
24        <label>Senha:</label>
25        <input class="entrada"/>
26
27        <br/>
28
29        <button class="botao">Enviar</button>
30      </form>
31    </div>
32  </body>
33
34  </html>
```

Fonte: Autoria Própria, 2025

No código HTML, conectamos o arquivo CSS através da tag “<link>”, permitindo com que a estilização que definimos seja aplicada aos elementos construídos na tela.

Figura 11 — Exemplo Formulário em CSS

A imagem mostra um formulário web estilizado com CSS. O formulário é contido dentro de um container azul claro com cantos arredondados. No topo, o título "Formulário" está em uma fonte grande e preta. Abaixo do título, há três campos de entrada de texto, cada um precedido por um rótulo: "Nome:", "Email:" e "Senha:". Os campos de entrada são retângulos brancos com bordas arredondadas. No rodapé do formulário, há um botão "Enviar" com um fundo cinza claro e o texto em preto.

Fonte: Autoria Própria, 2025

2.2.6 JavaScript

Baseado em David Flanagan (2013), o JavaScript (JS) é utilizado com frequência em sites e navegadores modernos — sendo uma das linguagens que o desenvolvedor web deve conhecer, já que especifica o comportamento das páginas na internet, dando movimento.

Na programação, é uma linguagem dinâmica, portanto sua execução é interpretada por interpretadores em tempo de execução, sem necessariamente uma compilação prévia (CONTRIBUTORS, 2021 apud MORÓRÓ, 2024).

Na construção de um site, o JavaScript é representado pela tag “<script>”, geralmente localizada dentro da tag “<head>”, permitindo a associação de arquivos externos ao HTML, mas sempre apontam para arquivos em JS (PUREWAL, 2014).

Agora, adicionaremos esses conceitos no formulário anterior de HTML e CSS, para adicionar funcionalidades:

Figura 12 — Exemplo Código em JS

```
<script>
  function mensagemDeEnvio() {
    var nome = document.getElementById("entradaNome");
    var email = document.getElementById("entradaEmail");
    var senha = document.getElementById("entradaSenha");

    alert(
      "Nome do usuário=" + nome.value +
      "\nEmail do usuário=" + email.value +
      "\nSenha do usuário=" + senha.value
    );
  }
</script>
```

Fonte: Autoria Própria, 2025

Linha 1: Inicia a tag “<script>”, onde fica o conteúdo do código de JavaScript.

Linha 2: É definida uma função chamada “mensagemDeEnvio”, que será executada ao clicar no botão.

Linha 3 à 5: São criadas variáveis que armazenam o conteúdo dos elementos com os Ids especificados em cada linha, como o “entradaNome”.

Linha 7: Quando o código for executado, a tag “alert(” abrirá uma caixa de alerta no seu navegador e mostrará como uma mensagem o conteúdo dentro da tag.

Linha 8 à 10: Mostra o texto escrito concatenado ao valor armazenado da variável, o qual é obtido através do “.value”. O “\n” serve para quebrar a linha antes do texto aparecer, impedindo que tudo fique na mesma linha.

Figura 13 — Exemplo Código em JS e HTML

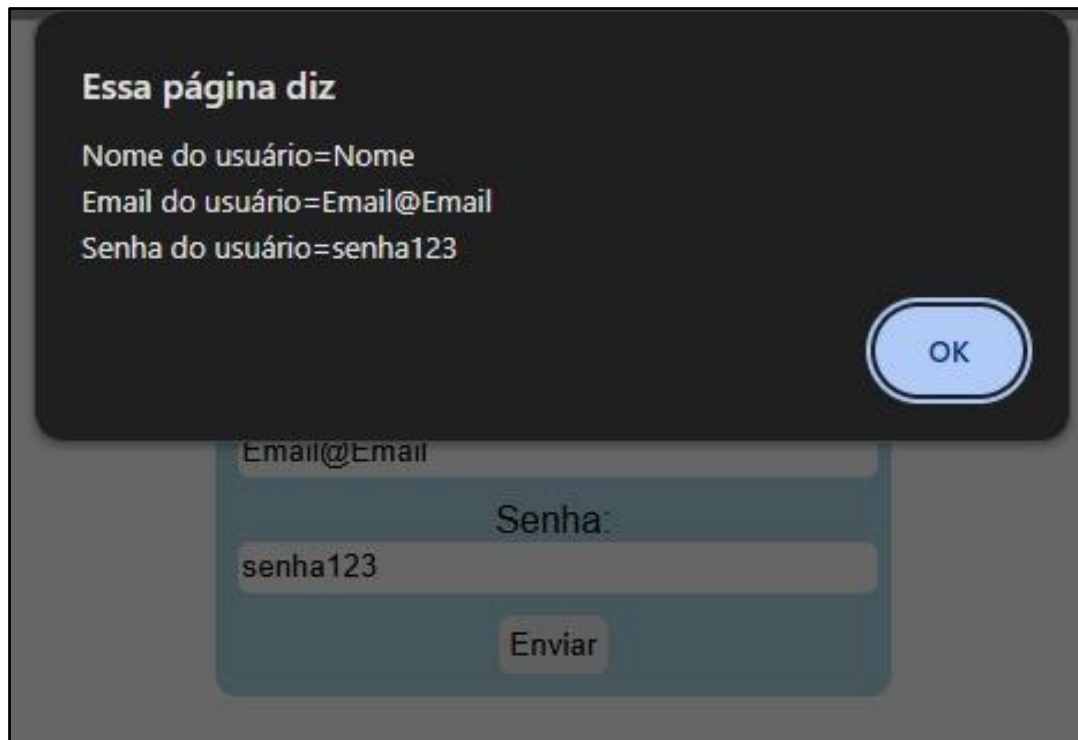
```
<button type="button" onclick="mensagemDeEnvio()" class="botao">Enviar</button>
```

Fonte: Autoria Própria, 2025

Para que o botão realmente execute a função, colocamos no “onclick=” a nossa função, nesse caso a “mensagemDeEnvio()”, conectando o botão com a mesma e tornando ela executável.

A seguir, veja a caixa de alerta que a função abre quando apertamos o botão:

Figura 14 — Exemplo Formulário em JS



Fonte: Autoria Própria, 2025

2.2.7 React

React é um framework de JavaScript fundamentado pela Meta, sendo extremamente popular, de código aberto e frequentemente atualizada, seu principal foco é construir interfaces de usuário com atualizações rápidas e reativas (NEVES, 2023).

A construção de telas pelo React é feita com componentes e elementos, tendo uma segmentação maior do código. Suas interações são baseadas em estados e eventos, assim gerando códigos reativos e dinâmicos (AZZOLINI, 2021).

React é especificamente utilizado para criar interfaces de usuário com interações reativas e de atualização imediata, seu uso se tornou imensamente popular no desenvolvimento web, e, hoje, é um dos frameworks mais utilizados (SILVA; OLIVEIRA, 2023).

Abaixo temos a construção de um formulário em React para exemplificar os conceitos:

Figura 15 — Exemplo código em React

```
1  import './App.css'
2
3  function App() {
4
5    return (
6      <>
7
8        <div className='form'>
9          <h1>Formulário</h1>
10         <hr />
11         <form>
12           <label>Nome:</label>
13           <input />
14
15           <br />
16
17           <label>Email:</label>
18           <input />
19
20           <br />
21
22           <label>Email:</label>
23           <input />
24
25           <br />
26
27           <button>Enviar</button>
28         </form>
29       </div>
30     </>
31   )
32 }
33
34
35 export default App
```

Fonte: Autoria Própria, 2025

O código acima é muito parecido com o código do formulário simples feito em HTML. Além de parte da estrutura ser diferente, como a ausência das tags “<head>” e “<body>”, o CSS é conectado através da tag “import” e a construção da página é feita dentro de uma função, neste caso a nomeada de “App()”.

Figura 16 — Exemplo Formulário em React



Formulário

Nome:

Email:

Email:

Enviar

Fonte: Autoria Própria, 2025

2.2.8 React Native

Embora React Native tenha suas semelhanças com React, isso não descarta suas diferenças, a linguagem é executada com o JavaScript Core com uma ponte para tradução do código para renderização em qualquer sistema operacional móvel (SILVA; DE SOUZA, 2019).

Como apontado por Bruna Escudelario e Diego Pinho, React Native é um framework que permite aplicativos híbridos mobile sejam desenvolvidos com as mais sofisticadas ferramentas front-end, com uma compatibilidade com iOS e Android (ESCUDELARIO; PINHO, 2021).

Segundo Franklyn Seabra Rogério Bezerra, React Native não funciona assim como um sistema web em WebView, e sim como um elemento nativo do sistema do

dispositivo, assim se desempenhando como um aplicativo na camada nativa (BEZERRA, 2021).

A diante, adicionaremos o framework React Native no formulário anterior, veja abaixo:

Figura 17 — Exemplo de Código em React Native

```
1  import { FormBasicExample } from '@components/FormBasicExample';
2
3  export default function FormScreen() {
4    |   return <FormBasicExample />;
5  }
```

Fonte: Autoria Própria, 2025

Essa tela existe para carregar e mostrar o componente “FormBasicExample”, onde está localizado o formulário básico que será exibido. Esse código pode ser bem útil para a organização do projeto, principalmente quando estamos utilizando rotas.

Figura 18 — Exemplo de Código em React Native

```
1  import { useState } from 'react';
2  import { Alert, StyleSheet, TextInput, View } from 'react-native';
3
4  import { ThemedText } from '@components/ThemedText';
5  import { ThemedView } from '@components/ThemedView';
6
```

Fonte: Autoria Própria, 2025

Na imagem acima, estão sendo feitas importações para o componente de React Native, o preparando para a utilização de estados, criação de alertas, campos de texto e layouts, aplicação de estilos e utilização de componentes personalizados.

Figura 19 — Exemplo de Código em React Native

```
16   return (  
17     <ThemedView style={styles.caixaFormulario}>  
18       <ThemedText type="title">Formulário</ThemedText>  
19       <View style={styles.hr} />  
20       <View>  
21         <ThemedText>Nome:</ThemedText>  
22         <TextInput  
23           style={styles.entrada}  
24           value={nome}  
25           onChangeText={setNome}  
26           placeholder="Digite seu nome"  
27         />  
28  
29         <ThemedText>Email:</ThemedText>  
30         <TextInput  
31           style={styles.entrada}  
32           value={email}  
33           onChangeText={setEmail}  
34           placeholder="Digite seu email"  
35           keyboardType="email-address"  
36           autoCapitalize="none"  
37         />  
38       </View>  
39     </ThemedView>  
40   );  
41 }
```

Fonte: Autoria Própria, 2025

O código demonstrado define os elementos da interface do formulário, montando os campos “nome” e “email”, que são estilizados com tema, ligados a estados internos que são atualizados em tempo real conforme o usuário digita.

Figura 20 — Exemplo de Código em React Native

```
export function FormBasicExample() {  
  const [nome, setNome] = useState('');  
  const [email, setEmail] = useState('');  
  const [senha, setSenha] = useState('');  
  
  function mensagemDeEnvio() {  
    Alert.alert('Formulário enviado', `Nome: ${nome}\nEmail: ${email}\nSenha: ${senha}`);  
  }  
}
```

Fonte: Autoria Própria, 2025

Esse código cria três estados para capturar o que o usuário digita no formulário e uma função que exibe esses dados em um alerta quando o formulário é enviado.

Figura 21 — exemplo de Código em React Native

```
38
39     <ThemedText>Senha:</ThemedText>
40     <TextInput
41       style={styles.entrada}
42       value={senha}
43       onChangeText={setSenha}
44       placeholder="Digite sua senha"
45       secureTextEntry
46     />
47
48     <View style={{ marginTop: 10 }}>
49       <ThemedText
50         style={styles.botao}
51         onPress={mensagemDeEnvio}
52         selectable={false}
53         accessibilityRole="button"
54       >
55         Enviar
56       </ThemedText>
57     </View>
58   </View>
59 </ThemedView>
60 );
61 }
```

Fonte: Autoria Própria, 2025

Esse trecho adiciona o campo de senha ao formulário e cria um botão estilizado que, ao ser pressionado, chama a função “mensagemDeEnvio” para mostrar os dados digitados em um alerta.

Figura 22 — Exemplo de Código em React Native

```
62
63  const styles = StyleSheet.create({
64    caixaFormulario: {
65      width: '90%',
66      maxWidth: 400,
67      padding: 16,
68      borderRadius: 10,
69      backgroundColor: 'lightblue',
70      alignSelf: 'center',
71      marginTop: 32,
72      textAlign: 'center',
73    },
74    hr: {
75      borderBottomWidth: StyleSheet.hairlineWidth,
76      borderBottomColor: '#888',
77      marginVertical: 12,
78    },
79    entrada: {
80      borderWidth: 0,
81      marginBottom: 10,
82      borderRadius: 5,
83      width: '100%',
84      height: 40,
85      backgroundColor: '#fff',
86      paddingHorizontal: 8,
87    },
88    botao: {
89      backgroundColor: '■ #87ceeb',
90      borderRadius: 5,
91      padding: 10,
92      textAlign: 'center',
93      color: '#222',
94      fontWeight: 'bold',
95      overflow: 'hidden',
96    },
97  });
```

Fonte: Autoria Própria, 2025

Este código é responsável pela estilização dos componentes da página, criando estilos para o container principal, os campos de texto e o botão.

Figura 23 — Exemplo Formulário em React Native

The image shows a mobile application interface with a light blue header bar. The status bar at the top displays the time 22:45, Bluetooth, Wi-Fi, 5G signal, and a 100% battery icon. The main content area has a light blue background with a white rounded rectangle containing the form. The form is titled 'Formulário' in bold black text. Below the title are three input fields labeled 'Nome:', 'Email:', and 'Senha:'. At the bottom of the form is a blue button with the text 'Enviar'. The bottom of the screen features a white navigation bar with three icons: a house icon labeled 'Ho...', a right-pointing arrow icon labeled 'Explore', and a right-pointing arrow icon labeled 'Formulá...'. A horizontal line is positioned above the navigation bar.

Fonte: Autoria Própria, 2025

2.2.9 TypeScript

TypeScript é uma linguagem de programação desenvolvida com base no JavaScript padrão, adicionando recursos avançados para a linguagem. Foi projetada com o principal intuito de solucionar limitações que o JavaScript possui (MORÓRÓ, 2024).

De acordo com Francisco Souza, Edilson Lima e Elda Caridade (2022), essa linguagem é fortemente tipada e mais escalável que o JS — muito utilizada por frameworks de FrontEnd — algo benéfico para desenvolvedores FullStack.

Assim, demonstraremos a seguir a implementação do TypeScript no formulário de exemplo:

Figura 24 — Exemplo Código em TS

```
1  function mensagemDeEnvio(): void {  
2      const nome = document.getElementById("entradaNome") as HTMLInputElement;  
3      const email = document.getElementById("entradaEmail") as HTMLInputElement;  
4      const senha = document.getElementById("entradaSenha") as HTMLInputElement;  
5  
6      alert(  
7          "Nome do usuário=" + nome.value +  
8          "\nEmail do usuário=" + email.value +  
9          "\nSenha do usuário=" + senha.value  
10     );  
11 }  
12
```

Fonte: Autoria Própria, 2025

Linha 1: Declaramos uma função chamada “mensagemDeEnvio”. A tag “void” significa que esta função não retorna nenhum valor.

Linha 2 à 4: São declaradas as constantes “nome”, “senha” e “email”, que parecido com o JavaScript, pegam o valor dos elementos do HTML através dos seus Ids. A tag “HTMLInputElement” presente nas linhas é uma asserção de tipo no TypeScript, onde diz que aquele elemento é um campo de entrada de texto.

Linha 6: A tag “alert(” funciona em TypeScript do mesmo jeito que no JavaScript, abrindo uma caixa de alerta quando executada.

Linha 7 a 9: Assim como no JavaScript, apresenta o texto armazenado na constante.

2.2.10 Banco de dados

De acordo com C.J. Date (2004), um sistema de Banco de Dados é apenas uma coleção computadorizada de gerenciamento de registros, servindo como um repositório para arquivos de dados digitais.

Os bancos de dados possuem um grande impacto em áreas que utilizam de computadores, pois sua capacidade de abrigar dados de qualquer tamanho e complexidade torna os bancos benéficos ao trabalho (ELMASRI; NAVATHE, 2011).

O banco de dados é quem molda a interface, afirma Lev Manovich (2015); a estrutura de armazenamento de dados é quem dita como deverá ser o funcionamento de uma página, definindo os elementos que precisaram ser criados para aquela tela.

2.2.11 Firebase

O Firebase é uma plataforma, que possibilita desfrutar de serviços em nuvem para a construção do BackEnd, seja de um sistema web ou uma aplicação móvel (FIREBASE, 2021 apud RODRIGUES, 2021).

O uso dessa ferramenta traz diversos benefícios — uma organização que permite fácil entendimento, escalabilidade para suportar grandes demandas, além de possuir formas de otimizar o desenvolvimento de softwares (ALURA, 2024)

Entre os serviços oferecidos pelo Firebase, estão incluídos um serviço de banco de dados chamado de Cloud FireStore, a autenticação de usuários, envio de notificações e armazenamento de arquivos, como dito por Amanda Anjolin Rodrigues (2021).

2.2.12 Construct

É um motor gráfico para construção de jogos em duas dimensões baseado em HTML, que possibilita executá-lo em aplicações web, de forma nativa ou por navegadores (CAPTERRA, 2021 apud CARVALHO; ARANHA; CARDIA NETO, 2022).

Como notado por Rodrigo Ribeiro Silva, Luis Rivero e Rodrigo dos Santos, Construct é muito vantajoso para um desenvolvimento rápido dos jogos — pouco complexo, acessível, com uma comunidade de suporte e plataformas flexíveis para disponibilizar os jogos (SILVA; RIVERO; SANTOS, 2021).

De muitas ferramentas de desenvolvimento de jogos, essa mostra uma simplicidade e didática na construção, facilitando assim o desenvolvimento e implementação de ideias na criação de videojogos (ALVES; AZEVEDO; COSTA JUNIOR, 2019).

O Construct se mostra mais eficaz em prototipação e facilitando a elaboração em comparação a similares, tendo recursos mais adequados para uma equipe onde nem todos possuem o mesmo conhecimento técnico (CARVALHO; ARANHA; CARDIA NETO, 2022).

2.2.13 C++

A linguagem de programação C, a qual serviu de base para a construção da linguagem C++, foi inicialmente criada para conseguir ser interpretada por máquinas de forma eficiente e rápida (HORSTMANN, 2008).

Como descrito por Diego Rodrigues, C++ surgiu como uma extensão da linguagem C, incluindo estruturas de programação orientada a objeto junto com o controle de baixo nível do C, formando assim uma linguagem muito versátil e poderosa (RODRIGUES, 2024).

Conforme Chen (2010 apud MARTINS, 2013) e Prechelt (2000, p. 29 apud MARTINS, 2013), concluiu-se que as linguagens C e C++ desempenham significativamente melhor em tempo real e economizam memória quando comparadas com outras linguagens.

Abaixo você pode ver um código em C++, criado para ligar e desligar um LED ao apertar de um botão, a estrutura utilizada é específica para funcionar em microcontroladores.

Com isso em mente, construímos este exemplo abaixo, para possibilitar a visibilidade do código em C++:

Figura 25 — Exemplo de código em C++

```
1  const int botao = 2; //Define a porta para o botão
2  const int led = 13; //Define a porta para o LED
3
4  int estadoDoBotao = 0;
5
6  void setup() {
7      pinMode(botao, INPUT);
8      pinMode(led, OUTPUT);
9  }
10
11 void loop() {
12     estadoDoBotao = digitalRead(botao);
13
14     if (estadoDoBotao == HIGH) {
15         digitalWrite(led, HIGH); //Liga o LED
16     }
17     else {
18         digitalWrite(led, LOW); //Desliga o LED
19     }
20 }
```

Fonte: Autoria Própria, 2025

Linha 1: É declarada uma constante de tipo inteiro armazenando o valor 2, de nome “botao”, esta linha se refere à porta do microcontrolador onde o botão está conectado.

Linha 2: É declarada uma constante de tipo inteiro armazenando o valor 13, de nome “led”, esta linha se refere à porta do microcontrolador onde o LED está conectado.

Linha 4: É declarada uma variável de tipo inteiro inicializada com o valor 0, de nome “estadoDoBotao”, esta linha receberá o estado atual do botão.

Linha 6: É criada uma função de nome “setup”, esta função é imediatamente executada ao código ser interpretado pela máquina.

Linha 7: A porta que o botão está conectado é definido como uma entrada.

Linha 8: A porta que o LED está conectado é definido como uma saída.

Linha 11: É definida uma função de nome “loop”, esta função será executada repetidamente e indeterminadamente.

Linha 12: A variável “estadoDoBotao” recebe o valor do botão.

Linha 14: É criado um operador lógico “if”, o código em seu interior será executado apenas se o botão estiver ligado.

Linha 15: Liga o LED.

Linha 17: É criado um operador lógico “else”, o código em seu interior será executado apenas se as condições para o último operador lógico “if” não forem atingidas.

Linha 18: Desliga o LED.

2.2.14 Internet of Things (IOT)

A internet das coisas — é um conceito, onde dispositivos físicos, por meio de sensores, se conectam à internet — sendo um facilitador no cotidiano atual, defende Eduardo (2018).

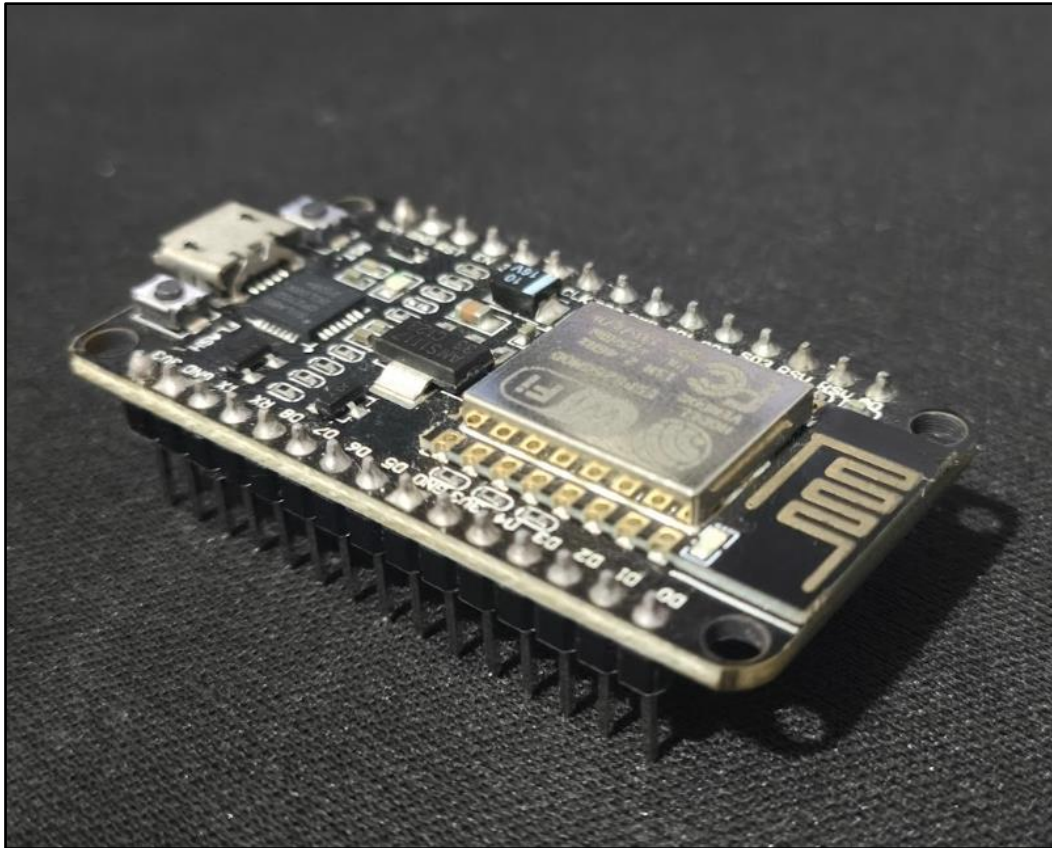
Segundo Corneto e Márcia (2024), a junção vinculada ao termo IoT, interage com o ambiente e com outros dispositivos, podendo coletar uma gama de dados, o que serve como base e meio para a criação de outros sistemas.

2.2.15 ESP32

CPU (Unidade Central de Processamento) e memória em uma pequena placa de circuito impresso que contém diversas capacidades de comunicação sem fio (BERTOLETI, 2019).

Segundo José V. S. Moraes, ESP32 veio como uma versão mais poderosa do microcontrolador, já consolidado no mercado, ESP8266, tendo dois núcleos de processamento, mais periféricos e Bluetooth, continuando com um preço competitivo (MORAIS, 2023).

Figura 26 — Microcontrolador ESP8266



Fonte: Autoria Própria, 2025

De acordo com Allyson Nascimento, o microcontrolador ESP32, especialmente ESPWROOM-32, possui suporte para Wi-Fi, BLE (Bluetooth Low Energy) e Ethernet, além de possuir grande eficiência energética (NASCIMENTO *et. al*, 2023).

2.2.16 Modelagem 3D

A modelagem, como explicado por Priscila Argoud (2024) — é uma técnica de recriar o mundo real de forma virtual, criando objetos tridimensionais (3D) nos meios digitais mimetizando a realidade — aplicando conceitos físicos e geoespaciais.

Contudo, por meio da modelagem se busca o detalhamento durante a construção, possibilitando uma imersão ao usuário — trazendo o virtual para algo mais concreto e agradável (VILELA; LOPES; LIMA, 2015).

2.2.17 Impressão 3D

Como dito por Moisés Miranda e Gustavo Henrique (2020), a impressão tridimensional — é a capacidade de modelar fisicamente, objetos e seres inanimados, de diversos tipos — sejam eles detalhados, simples, robustos, grandes ou pequenos.

É explicado por Silva *et. al* (2020), que o processo de impressão é feito em camadas — mais especificamente fatias baseadas no formato do modelo que está sendo impresso — que são adicionadas uma junta da outra, criando espessura e relevo.

Afirma Leonardo Gomez, que a impressão 3D, atrai alta atenção de quem a manuseia, apesar de complexas, ao transformar os modelos escaneados em atrações físicas, estimula bastante a criatividade e pensamento livre.

3 DESENVOLVIMENTO

4 CONSIDERAÇÕES FINAIS

REFERÊNCIAS

ACETI, L. B.; BRITO, L. T.; OLIVEIRA, D. C. M.; PINA, D. B.; KUNSTMANN, L. N. O. **Desenvolvimento de uma Aplicação Web para Instrumentação de Scripts para a Ferramenta DfAnalyzer**. 2023. Trabalho de Conclusão de Curso (Bacharelado em Sistemas de Informação) - Universidade Federal Fluminense, Niterói, 2023.

ALURA. 2024. **O que é Firebase? Para que serve, principais característica e um guia dessa ferramenta Google**. Disponível em: <https://www.alura.com.br/artigos/firebase?srsltid=AfmBOopem7O3TvcbgX7pGKqJmj6aesJI7WIJtDFoEtQfEpk0PCVOwqyC>. Acesso em: 28 abr. 2025.

ALVES, L. F. D.; AZEVEDO, V. N. DE L.; COSTA JUNIOR, A. DE O. Construct 2: Oficinas Formativas Para a Criação de Jogos Educacionais Digitais. **III Simpósio Ibero-Americano de Tecnologias Educacionais**, jun. 2019.

ANACLETO CHICCA JUNIOR, N.; GOMEZ CASTILLO, L. **Os desafios em utilizar a impressão 3D no processo ensino-aprendizagem de design**. RENOTE, Porto Alegre, v. 16, n. 1, 2018. DOI: 10.22456/1679-1916.86040. Disponível em: <https://seer.ufrgs.br/index.php/renote/article/view/86040>. Acesso em: 29 abr. 2025.

APOCALYPSE, Marcos; JORENTE, Maria. **O MÉTODO DESIGN THINKING E A PESQUISA EM CIÊNCIA DA INFORMAÇÃO**. Encontros Bibli, Florianópolis, v.27, n.1, p. 1-21, nov. 2022.

ARGOUD, P. **Modelagem 3D**. São Paulo: SENAC, 2024. Disponível em: <https://play.google.com/books/reader?id=iCgGEQAAQBAJ&pg=GBS.PP1&hl=ptBR>. Acesso em: 29 abr. 2025. E-book.

AZZOLINI, M. G. **Derivação de um estilo arquitetural para o front-end de sistemas baseados em React.js com base em projetos Open Source**. Ufrgs.br, 2021.

BEZERRA, Franklyn Seabra Rogério. **Desenvolvimento Nativo vs Ionic vs React Native: uma análise comparativa do suporte à acessibilidade em Android**. 2021. 25 f. Trabalho de Conclusão de Curso (Graduação em Computação)-Universidade Federal do Ceará, Fortaleza, 2021.

BERTOLETI, P. **Projetos com ESP32 e LoRa**. São Paulo: INSTITUTO NEWTON C. BRAGA, 2019.

BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. **UML: guia do usuário**. 2. ed. Rio de Janeiro: Elsevier, 2006.

CARVALHO, L.; ARANHA, G.; CARDIA NETO, J. B. . JOGOS DIGITAIS. **Interface Tecnológica**, v.18, n.2, p. 264-267, 14 maio, 2022.

DATE, C. J. Tradução de Daniel Vieira. **Introdução a sistemas de bancos de dados**. Rio de Janeiro: Elsevier Brasil, 2004.

ELMASRI, Ramez; NAVATHE, Shamkant B. **SISTEMAS DE BANCO DE DADOS**. 6. ed. São Paulo: Addison Wesley, 2011.

ESCUDELARIO, B.; PINHO, D. **React Native**. São Paulo, SP: Casa do Código, 2021.

ESTADO DE MINAS. **Uso excessivo de telas pode causar demência digital, diz neurocirurgião**. 6 out. 2023. Disponível em: https://www.em.com.br/app/noticia/saude-e-bem-viver/2023/10/06/interna_bem_viver,1572765/uso-excessivo-de-telas-pode-causar-demencia-digital-diz-neurocirurgiao.shtml. Acesso em: 19 abr. 2025.

FEDERAL DE SANTA, U. et al. Revista eletrônica de biblioteconomia e ciência da informação: O MÉTODO DESIGN THINKING E A PESQUISA EM CIÊNCIA DA INFORMAÇÃO Encontros Bibli: revista eletrônica de biblioteconomia e ciência da informação. v. 27, p. 1–21, 2022.

FLANAGAN, D. Tradução de João Eduardo Nóbrega Tortello. **JavaScript: o Guia Definitivo**. 6. ed. Porto Alegre: Bookman, 2013.

FLATSCHART, Fábio. **HTML 5: Embarque Imediato**. Rio de Janeiro, RJ: Brasport, 2011.

FOWLER, Martin. **UML essencial: um breve guia para a linguagem-padrão de modelagem de objetos**. 3. ed. Porto Alegre: Bookman, 2005.

FREIRE, Paulo. **Pedagogia da autonomia: saberes necessários à prática educativa**. São Paulo: Paz e Terra, 1996.

FROBOESE DA SILVA, Eloar; MENIN DA SILVA, Lucas; GONÇALVES DEON, Vinícius; TOSO, Marcelo André. IMPRESSÃO 3D APLICADA À TECNOLOGIA ASSISTIVA. **Destaques Acadêmicos**, [S. l.], v. 12, n. 4, 2020. DOI: 10.22410/issn.2176-3070.v12i4a2020.2657. Disponível em: <https://www.univates.br/revistas/index.php/destaques/article/view/2657>. Acesso em: 29 abr. 2025.

GUEDES, Gilleanes T. A. **UML 2: uma abordagem prática**. 3.ed. São Paulo: Novatec, 2009.

G1. **'Cérebro podre'**: entenda o que é 'brain rot', expressão do ano eleita pelo Dicionário Oxford. G1 Educação, 02 dez. 2024. Disponível em:

<https://g1.globo.com/educacao/noticia/2024/12/02/entenda-o-que-e-brain-rot-cerebro-podre-expressao-do-ano-eleita-pelo-dicionario-oxford.ghtml>. Acesso em: 12 maio 2025.

HORSTMANN, Cay. **Conceitos de computação com o essencial de C++**. Tradução de Carlos Arthur Lang Lisbôa e Mária Lúcia Blanck Lisbôa. 3ªed. Porto Alegre: ARTMED, p. 30- 2008. Disponível em: https://play.google.com/store/books/details?id=qSn8wEMsolMC&rdid=bookhttps://play.google.com/store/books/details?id=qSn8wEMsolMC&rdid=book-qSn8wEMsolMC&rdot=1&source=gbv_vpt_read&pcampaignid=books_booksearch_viewportqSn8wEMsolMC&rdot=1&source=gbv_vpt_read&pcampaignid=books_booksearch_viewport. Acesso em 26 Abr. 2025. E-book.

JOBSTRAIBIZER, Flávia. **Criação de Sites CSS**. São Paulo, SP: Digerati Books, 2009.

LARMAN, Craig. **Utilizando UML e padrões**: uma introdução à análise e ao projeto orientados a objeto. Porto Alegre: Bookman, 2000.

MAGRANI, E. **A internet das coisas**. Rio de Janeiro: FGV Editora, 2018. 192 p.

MANOVICH, L. Banco de Dados. **Eco-Pós**, [S. l.], v. 18, n. 1, p. 7–26, 2015. Disponível em: https://ecopos.emnuvens.com.br/eco_pos/article/view/2366. Acesso em: 28 abr. 2025.

MARQUES, P. B.; CASTANHO, M. I. S. O que é a escola a partir do sentido construído por alunos. **Psicologia Escolar e Educacional**, v. 15, n. 1, jun. 2011. DOI: <https://doi.org/10.1590/S1413-85572011000100003>).

MARTINS, Mateus Rodrigues. **Desenvolvimento de uma biblioteca em C++ para modelagem e controle cinemático de robôs manipuladores usando quatérnios duais**. 2013. Trabalho de Conclusão de Curso (Bacharelado em Engenharia de Controle e Automação) — Universidade Federal de Minas Gerais, Belo Horizonte, 2013.

MILETTO, E. M.; DE CASTRO BERTAGNOLLI, S. **Desenvolvimento de Software II: introdução ao desenvolvimento web com HTML, CSS, JavaScript e PHP - eixo: informação e comunicação - Série Tekne**. Porto Alegre: Bookman Editora, 2014.

MIRANDA MORANDINI, M.; DEL VECHIO, G. H. IMPRESSÃO 3D, TIPOS E POSSIBILIDADES: uma revisão de suas características, processos, usos e tendências. **Interface Tecnológica**, Taquaritinga, SP, v. 17, n. 2, p. 67– 77, 2020. DOI: 10.31510/inf.v17i2.866. Disponível em: <https://revista.fatectq.edu.br/interfacetecnologica/article/view/866>. Acesso em: 29 abr. 2025.

MORAIS, J. V. S. **ESP32 com IDF**. São Paulo: Editora NCB, 2023.

MORORÓ, J. F. **Um estudo comparativo entre JavaScript e TypeScript**. 2024. 154 f. Trabalho de Conclusão de Curso (Graduação em Engenharia de Computação) – Campus de Sobral, Universidade Federal do Ceará, Sobral, 2024. Acesso em: 28 de Abril, 2025.

NASCIMENTO, A.; SOUSA, C.; OLIVEIRA, G.; OLIVEIRA, F.; CAMPOS, G. **Controle de Iluminação Através da Internet Utilizando as Tecnologias ReactJS, Firebase e ESP32**. Anais do Encontro de Computação do Oeste Potiguar ECOP/UFERSA (ISSN 2526-7574), [S. l.], v. 1, n. 6, p. 46–49, 2023. Disponível em: <https://periodicos.ufersa.edu.br/ecop/article/view/11835>. Acesso em: 26 abr. 2025.

NEVES.V. React: o que é, como funciona e um Guia dessa popular ferramenta JS. Alura (2023).

PAGANI, T. **Design thinking**. [s.l.] SENAC, p. 3–9, 2018. Disponível em: https://play.google.com/store/books/details?id=lz1MDwAAQBAJ&rdid=bookhttps://play.google.com/store/books/details?id=lz1MDwAAQBAJ&rdid=book-lz1MDwAAQBAJ&rdot=1&source=gbs_vpt_read&pcampaignid=books_booksearch_viewportlz1MDwAAQBAJ&rdot=1&source=gbs_vpt_read&pcampaignid=books_booksearch_viewport . Acesso em: 17 maio. 2025. E-book.

PARISOTTO, A. L. V.; RINALDI, R. P. **Ensino de língua materna**: dificuldades e necessidades formativas apontadas por professores na Educação Fundamental. Educação e Revista, n. 60, abr./jun. 2016.

PAULA, B. H. de. **Jogos digitais como artefatos pedagógicos**: o desenvolvimento de jogos digitais como estratégia educacional. 2015. Dissertação (Mestrado em Artes Visuais) - Instituto de Artes, Universidade Estadual de Campinas, Campinas, 2015.

PUREWAL, S. Tradução de Lúcia Kinoshita. **Aprendendo a desenvolver aplicações web**: Desenvolva rapidamente com as tecnologias JavaScript mais modernas. São Paulo: Novatec Editora, 2014.

RODRIGUES, Amanda Anjolin. **Desenvolvimento de aplicativo Android utilizando os serviços de firebase**, 2021. Trabalho de conclusão de curso (Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas) Faculdade de Tecnologia de São Paulo, São Paulo, SP, 2021.

RODRIGUES, D. **APRENDA C++ MODERNO**: Desenvolva Aplicações Performáticas com Recursos Avançados. Dos Fundamentos às Aplicações Práticas. [s.l.] StudioD21, 2024.

SCHEIDT, F. A. **Fundamentos de CSS**: criando design para sistemas web. Foz do Iguaçu: [s.n.], 2015.

SENA, S. de; SCHMIEGELOW, S. S.; PRADO, G. M. B. C. do; PERASSI, R.; FIALHO, F. A. P. **Aprendizagem baseada em jogos digitais**: a contribuição dos jogos epistêmicos na geração de novos conhecimentos. CINTED-UFRGS Novas Tecnologias na Educação, v. 14, n. 1, jul. 2016.

SILVA, Denys Alves; DE SOUZA, Caio Frias. Construção de App com React Native. **TECNOLOGIAS EM PROJEÇÃO**, [S. l.], v. 10, n. 1, p. 1–15, 2019.

SILVA, Eduardo Corneto; ESPEJO, Márcia M. S. B. Adoção da Internet das Coisas (IoT) na agropecuária: uma revisão sistemática sobre as possibilidades de adoção no ambiente produtivo rural brasileiro. **Interações**, Campo Grande, v. 25, n. 4, 2024.

SILVA, Maurício Samy. **HTML 5: A LINGUAGEM DE MARCAÇÃO QUE REVOLUCIONOU A WEB**. São Paulo, SP: Novatec Editora, 2011.

SILVA, João Augusto Marciano; OLIVEIRA, Cintia Carvalho. Análise avaliativa acerca do uso do React para programação web front-end. In: ENCONTRO DE ENSINO, PESQUISA E EXTENSÃO, 10., 2023, Patrocínio, MG. **Anais [...]**. Patrocínio, MG: IFTM, 2023.

SOUZA, Francisco Moreira Calado; LIMA, Edilson Carlos Silva; CARIDADE, Elda Regina de Sena. CRIANDO SISTEMA ESCALÁVEL DE AGENDAMENTOS UTILIZANDO TYPESCRIPT COM NESTJS NO BACKEND E NEXTJS NO FRONTEND. **Ibero-Americana de Humanidades, Ciências e Educação**, [S. l.], v. 8, n. 12, p. 43–57, 2022. DOI: 10.51891/rease.v8i12.7986.

SILVA, R. R.; RIVERO, L.; SANTOS, R. P. dos. ProgramSE: Um Jogo para Aprendizagem de Conceitos de Lógica de Programação. Revista Brasileira de Informática na Educação, [S. l.], v. 29, p. 301–330, 2021. DOI: 10.5753/rbie.2021.29.0.301.

TENENTE, Luiza. ***Em teste de habilidades de leitura entre crianças, Brasil ocupa 39º lugar em ranking com 43 países***. G1, 16 maio 2023. Disponível em: <https://g1.globo.com/educacao/noticia/2023/05/16/em-teste-de-habilidades-de-leitura-entre-criancas-brasil-ocupa-39o-lugar-em-ranking-com-43-paises.ghtml>. Acesso em: 10 ago. 2025.

VASCONCELLOS, M. S. de; CARVALHO, F. G. de; BARRETO, J. O.; ATELLA, G. C. As Várias Faces dos Jogos Digitais na Educação. **Informática na Educação: teoria & prática**, Porto Alegre, v. 20, n. 4, p. 203-218, ago. 2017.

VIANNA, M. et al. **Design Thinking: Inovação em Negócios**. 1. ed. Rio de Janeiro, RJ: MJV Press, 2012.

VILELA, João Paulo; LOPES, Ricardo; LIMA, Fernando. **Modelagem 3d de edifícios históricos: a influência do lod no processo de reconstrução virtual**. 2020. Trabalho de Graduação (Graduação em Engenharia Civil) - Universidade Estadual Paulista, Guaratinguetá, 2015.

